Lab 3 Report: Implementation of PD Controlled, LiDAR Guided, Wall-Following Robot with Safety Controller

Team 12

Alex Chen
Yaqi Li
Daniel Odenthal
Tina Zhang

Robotics: Science and Systems

March 13, 2026

1 Introduction

Autonomous driving requires the robot car to maintain a desired perpendicular distance to the wall at varying speeds while avoiding collisions with stationary and unexpected variables. In this lab, we developed a Proportional-Derivative (PD) controlled, LiDAR-guided, wall following program with a safety controller that allows the robot car to navigate corridors and turn corners, while maintaining a desired perpendicular distance and orientation. In addition, a safety controller prevents collisions by utilizing LiDAR data to estimate obstacles around the the car.

Our development process includes testing in both simulation and in the real world, defining the success of the program through metrics such as cross-track error, minimum stopping distance, minimum distance threshold, and the average deviation from the desired perpendicular distance. Our evaluation successfully demonstrated how desired qualities in simulation-based controller design can transfer to physical autonomous robots.

2 Technical Approach

2.1 Wall Follower

The goal of the wall follower program is to maintain a desired perpendicular distance between the car and a specified wall, in addition to making turns of 90 degrees or wider. At a high level, the program uses the LiDAR data to create a linear regression line that models the physical wall. The car then either follows a straight wall or turns a corner, depending on the environment. A PD controller corrects the orientation and distance of the car.

For our wall follower program to be deemed successful, it must meet the following requirements:

- Use LiDAR data to inform decisions
- Follow a straight wall, on either the left or the right side
- Make a 90 degree corner and wider turns
- Minimize oscillations via a PD controller
- Stop if driving id deemed dangerous by the safety controller

Our team had four different wall follower programs from the previous lab, and chose to use the one that scored the highest. The wall follower assumes that if there's an object in front, the car is approaching a corner and needs to make a turn. If the car doesn't see any object in front, then it continues to follow

the wall. These assumptions are made under the pre-defined map of the Stata basement.
At a high level, the algorithm does the following:

- Subscribes to the LiDAR data
- Filters the data based on which side of the wall is being followed
- Using the data, create a line of best fit using linear regression
- Calculates the car's perpendicular distance and orientation based on the line of best fit
- Sends a drive command for either moving straight or making a turn
- PD controller corrects orientation and minimizes oscillations

For the PD controller, the K_p and K_d were chosen based on a velocity of 1 $\frac{m}{s}$ in simulation. The K_p was chosen when the car displayed oscillations about the desired distance. The K_d term was then tuned to minimize said oscillations.

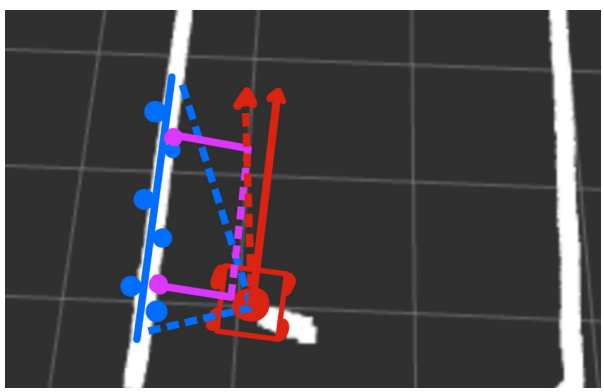

Figure 1: A RViz photo depicting LiDAR points (blue) creating a line of best fit. The car (red) then follows the line of best fit

2.2 Safety Controller

The goal of the safety controller is to maintain the safety of the vehicle by preventing collisions with known and sudden obstacles, such as wall or people. To accomplish this, we created a filter for the incoming drive commands from the wall follower before they reach the car. If the safety controller determines that the environment is safe, it passes the drive commands through. However, if the environment is deemed unsafe, the controller instead sends a stop command,

setting velocity to 0 $\frac{m}{s}$.

The filter, visualized in Figure 2, uses laser scan data from the LiDAR as its primary input. We convert the range and bearing measurements from the LiDAR into an x–y coordinate frame centered on the robot. The data is then masked to a rectangular region extending 0.25 m to either side of the car and from 0.05 m to 1.5 m in front of the vehicle. This masking removes "ghost" noise caused by wires or other parts of the car interfering with the LiDAR and also monitors the immediate sides of the vehicle to prevent both frontal collisions and side impacts when turning. We then compute a *safety distance* as a function of the high-level desired velocity,

$$d = K_1 \, v^{K_2} \tag{1}$$

where d is the safety distance, v is the commanded velocity, and K_1 and K_2 are system gains. The gains were experimentally tuned for the physical racecar operating in the range of one to three meters per second, resulting in values of $K_1 = 0.35$ and $K_2 = 0.25$.

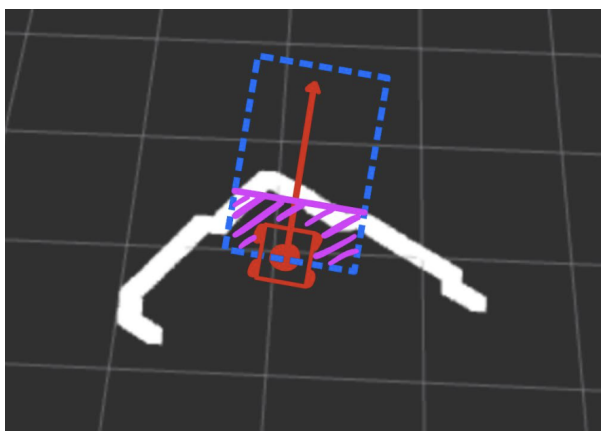

Figure 2: A screenshot of a RVIZ ROS2 Bag while the Safety Controller node was running. The white lines indicate the laser scan data collected by the racecar's LiDAR scanner. The red symbolizes the robot and its vector of travel. The blue box represents the area of laser scan data the node considers. The purple is the safety area, where any laser scan data within this space triggers the safety controller to stop the racecar.

If any laser scan measurement within the masked region returns a distance less than the computed safety distance, the safety controller publishes a stop command to the vehicle. The vehicle remains stopped until the controller determines that the car is moved to a safe environment.

3 Experimental Data and Evaluation

3.1 Technical Procedures

We evaluated the wall follower and safety controller independently in simulation using in-house test cases. All tests were run in the `racecar_simulator` environment and repeated $N = 3$ times each to ensure the consistency of results.

Wall Follower (Simulation). The `test_wall_follower` node teleports the car to a start pose, then on every LiDAR scan masks returns to a 30–100° window and fits a line through those points via linear regression to estimate perpendicular wall distance. After a 3-second convergence window, the node records CTE, minimum wall distance, speed, and steering angle, where

$$e = d_w - d^* \tag{2}$$

where e is CTE, d_w is perpendicular wall distance, and d^* is desired distance. A run is <u>successful</u> if mean absolute CTE $|\bar{e}| = \frac{1}{N}\sum_{i=1}^{N}|e_i| \leq 0.30$ m, where e_i is the CTE of the ith test run. Ten scenarios were tested: six core tests varying wall side, starting angle, and speed (up to 3 m/s), and four edge cases covering far/close starts, steep angled approach, and maximum speed (4 m/s).

Safety Controller (Simulation). The `test_safety_controller` node continuously publishes a full-speed, zero-steering command to simulate a navigation controller unaware of obstacles. The safety controller must detect the impending collision and publish a zero-speed command to halt the car. The node records whether the safety controller engaged, forward LiDAR distance at stop, and time to stop. A run is <u>successful</u> if the safety controller engages and stops the car such that $d_w \geq 0.05$ m when a stop is expected, and does not engage otherwise. Eight scenarios were tested covering head-on, angled (30°, 60°), high speed (3 m/s), parallel, reverse, very close, and corner approaches.

Physical Testing. Three experiments were conducted on the physical racecar with distances measured manually using a ruler. (1) *Straight wall following*: the car followed a straight wall at 2 m/s with desired distance 0.5 m from three starting distances, with wall distance sampled at fixed positions along the corridor. (2) *Corner tracking*: peak CTE was measured at 90° and 40° corners at 2 m/s. (3) *Safety stopping distance*: the car was driven straight at a wall from 3 m away at speeds from 1.0–3.0 m/s, measuring stopping distance across 3 runs per speed.

3.2 Results

Wall Follower (Simulation). All 10 scenarios passed across all 3 runs. Figure 3 shows mean CTE $\bar{e} \pm \sigma_e$ per scenario, with σ_e ranging from 0.12–0.19 m, which is below the 0.30 m threshold with low variance across runs. One of the worst case test scenarios was `high_speed_right` (4 m/s), shown in Figure 4,

where a spike near $t = 10$s approaches the error threshold at a corner. Nevertheless, the controller recovers and remains within threshold, which demonstrates the car's robustness at high speed.

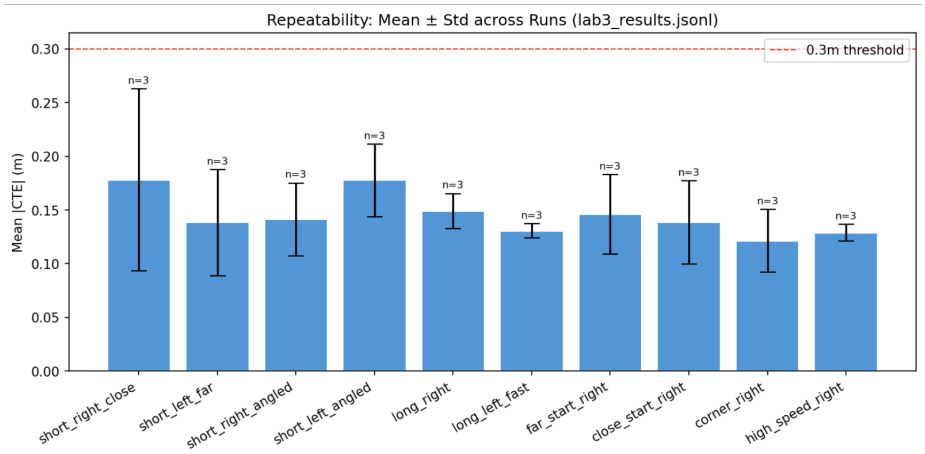

Figure 3: $\bar{e} \pm \sigma_e$ across 3 runs per wall follower scenario. All scenarios fall below the 0.30 m threshold (dashed).

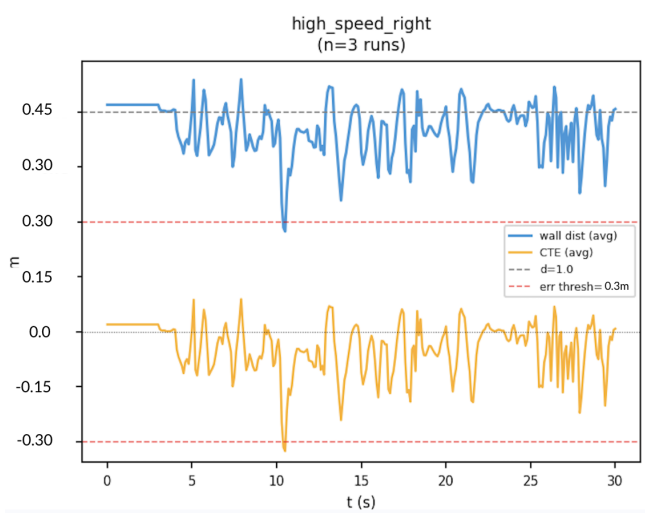

Figure 4: d_w and e over time for `high_speed_right` (4 m/s, averaged over 3 runs). Spike near $t = 10$ s corresponds to a corner.

Safety Controller (Simulation). All 8 scenarios passed across all 3 runs. Figure 5 confirms all scenarios where stop is expected is halted well above the 0.05 m threshold. Furthermore, Figure 5 shows the `angled_30_stop` test sce-

nario had the highest stopping time variance, which is expected since shallow approach angles take longer for the forward LiDAR cone to register the wall. All no-stop scenarios produced zero interventions, which confirms no false positives.

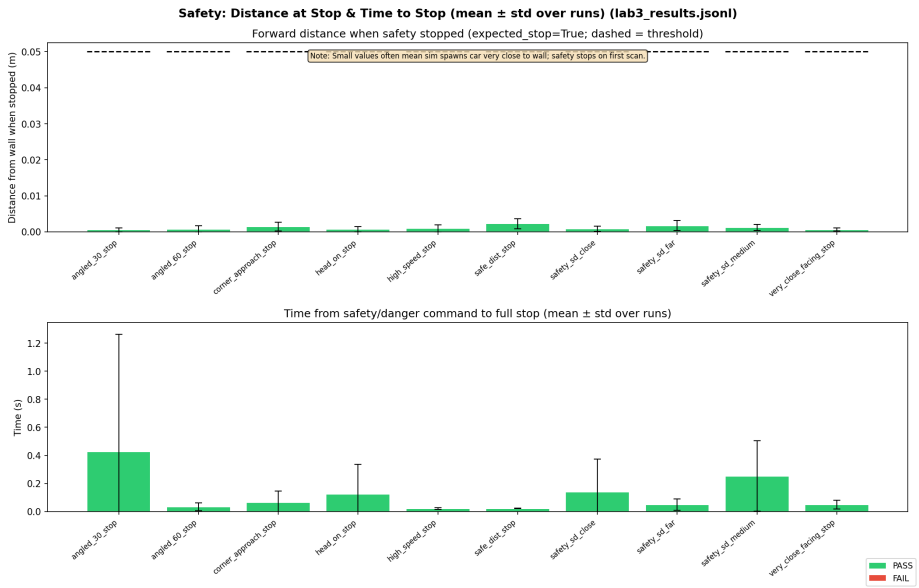

Figure 5: Safety controller simulation results: d_w at stop (top) and time to stop (bottom), mean $\pm$ std over 3 runs per scenario. All expected-stop scenarios satisfy $d_w \geq 0.05$ m (dashed).

Physical Testing. We conducted three physical tests to assess whether the car can a) follow a straight wall b) turn a corner and c) stop before a collision. Results are as follows:

- Straight Wall Folowing: $\bar{d}_w = 0.505$ m (1% deviation from $d^* = 0.5$ m) with $\sigma = 0.0093$ m across all starting positions, demonstrating accurate steady-state tracking.

Trial	Starting Perpendicular Distance	Datapoint at 1.524m	Datapoint at 3.048m	Datapoint at 4.572m	Average Distance to Wall
1	0.3048	0.505	0.506	0.495	0.502
2	0.3048	0.496	0.489	0.495	0.493
3	0.6096	0.505	0.485	0.503	0.497
4	0.6096	0.515	0.490	0.505	0.503
5	0.9144	0.540	0.503	0.517	0.520
6	0.9144	0.520	0.511	0.509	0.513

Table 1: Physical safety controller stopping distance at various velocities, starting 3 m from wall.

- Turning Corner: Corner tracking produced average peak deviations of 0.34 m (90°) and 0.41 m (40°), with overall $\sigma = 0.080$m, which is larger than straight tracking but no collisions occurred.

Trial	Wall Angle	Run 1 (m)	Run 2 (m)	Run 3 (m)
1	90°	0.34	0.36	0.33
2	40°	0.42	0.39	0.41

Table 2: Physical safety controller stopping distance at various velocities, starting 3 m from wall.

- Safety Stop: As shown in Table 3, safety stopping distance scaled consistently with speed, from 0.17 m at 1.0 m/s to 0.65 m at 3.0 m/s, with all runs stopping before the wall.

Trial	Velocity (m/s)	Run 1 (m)	Run 2 (m)	Run 3 (m)	Average (m)
1	1.0	0.17	0.17	0.19	0.17
2	1.5	0.22	0.20	0.22	0.21
3	2.0	0.20	0.16	0.23	0.20
4	2.5	0.34	0.33	0.35	0.34
5	3.0	0.64	0.65	0.65	0.65

Table 3: Physical safety controller stopping distance at various velocities, starting 3 m from wall.

4 Conclusion

Our wall follower and safety controller programs are proven successful through both simulation and physical tests. The system meets all of the design requirements. Despite the success, the wall follower and safety controller still can be

majorly improved. The wall follower has proven to be successful and robust on a straight wall and sharp corners. Test scenarios can be expanded to included curved and rugged walls, and even sharper corner turns. The safety controller algorithm calculates a stopping distance based on the car's velocity, but this approach can be improved to be more dynamic. In addition, further simulation and physical testing, data collection, and analysis, can create a more comprehensive report about the system, through which more specific improvements can be made. And finally, dynamic gains for different velocities can help improve the smoothness of the car's driving, and allow the car to drive at higher speeds.

Overall, our lab establishes a simple, stable, and safe wall following program that can reliably work in relatively clear areas, such as the Stata basement. we hope to build on this foundation by increasing controller robustness, expanding testing environments, and exploring more advanced navigation strategies that allows our vehicle to operate in more complex real-world environments.

5 Lessons Learned

Tina: This lab taught me how important it is to start early. Debugging testing and creating launch files took longer than expected. If we started earlier, we probably would've had more time for physical testing and PD tuning. I also learned to look deeper into why asystem behaves the way it does Eg: understanding that the CTE spike in the time-series graph comes from the LiDAR window briefly losing the wall during a turn instead of just accepting the spike as noise. Designing the simulation tests also encouraged me to think more about what scenarios would actually extensively test the controller (edge cases, false positives, etc.) and how to measure quantitatively performance.

Yaqi: Throughout this lab, I learned that starting the lab early is important since many parts, such as controller tuning, debugging, and integrating the wall follower with the safety controller, took longer than expected. I also learned that small changes in PD gains or safety thresholds could significantly affect stability so it is important for systematic testing. As for communication, I learned that it is important to keep clear communication with team members to regularly discuss and update everyone's progress to ensure that every member understands other parts of the lab. In addition, we should schedule physical testing sessions early to help avoid time conflicts and ensure enough time for adjustment. Finally, I learned the importance of defining clear success criteria before starting to implement the algorithm. Establishing specific performance goals early on would provide a clearer standard for evaluation metrics.

Daniel: This lab has taught me a lot. Technically, it was the first time I have ever done collaborative coding and that carries a pretty large learning curve. I had to spend a lot of time learning the software and the commands before I could even start trying to actually tackle the lab problem. I think this

will be better in future labs as I now have the essential building blocks I need to succeed. In terms of collaboration, I enjoyed working with my team members in person to tackle the physical problems of the robot, such as parameter tuning and testing, but it took up a lot of time. I think in future labs, it would be better to have a clear outline of tests for the robot as well as a specific timeline for how many minutes and how long we plan to spend on each test. In terms of communication, I think my team did a good job of setting expectations and holding each other to those standards. I think the greatest struggle was finding time to meet in person because of everyone's differing schedules.

Alex: Two of the most important lessons I learned from this lab is to start early and communicate clearly. Testing the system took longer than we expected, and debugging the system also added to the time. In addition, not having clear goals and task division in the start of the lab ate into our productivity, but that's an area we will be improving on in the future. In terms of technical skills, I feel more confident with using ROS2 across different environments and developing clear testing procedures. During the testing process, I also gained insight into the fact that there is not enough time to perfect every detail, and that as a team we will have to establish what is necessary, and what is a bonus. Overall, I enjoyed working on this lab, and I believe that we are off to a really good start.